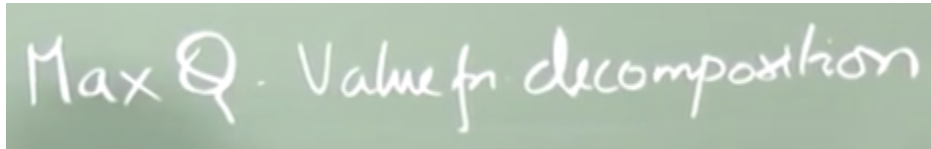


w11L60 MAXQ

Reference: [T. G. Dietterich. 2000. Hierarchical Reinforcement Learning with the MAXQ Value Function Decomposition.](#)

3rd of 3 popular

- MAXQ - original paper



I leave out value fn decomposition

But will cover briefly - for benefit of two students and in Deep Learning era!

REF: MAXQ - journal paper is ideal - how to write good paper

Mini-phd thesis! -> just read it, take good message!

Decompose problem in set of sub tasks



Example - -Taxi Domain - - 5x5 grid world

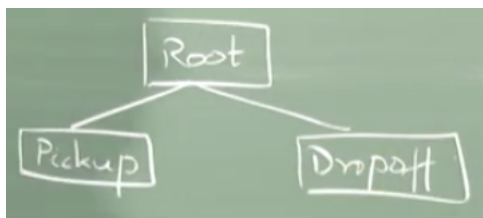
Pickup passenger and drop at 4 places r, g, b, y - gets reward +10

Every move = -1

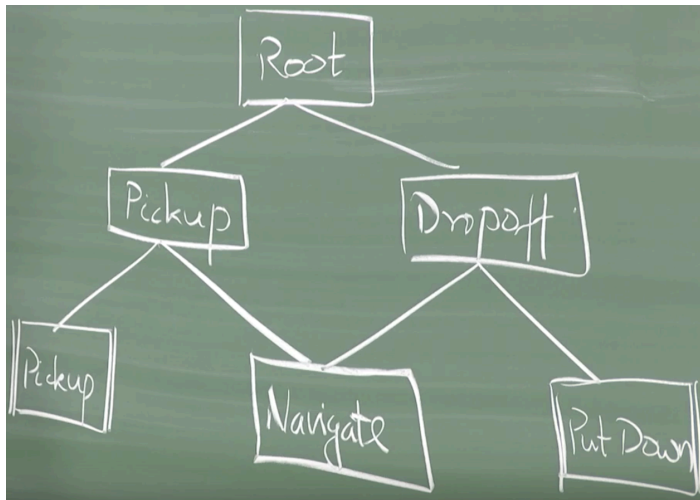
Randomly passenger pop up

Lot of repeated sub-structure

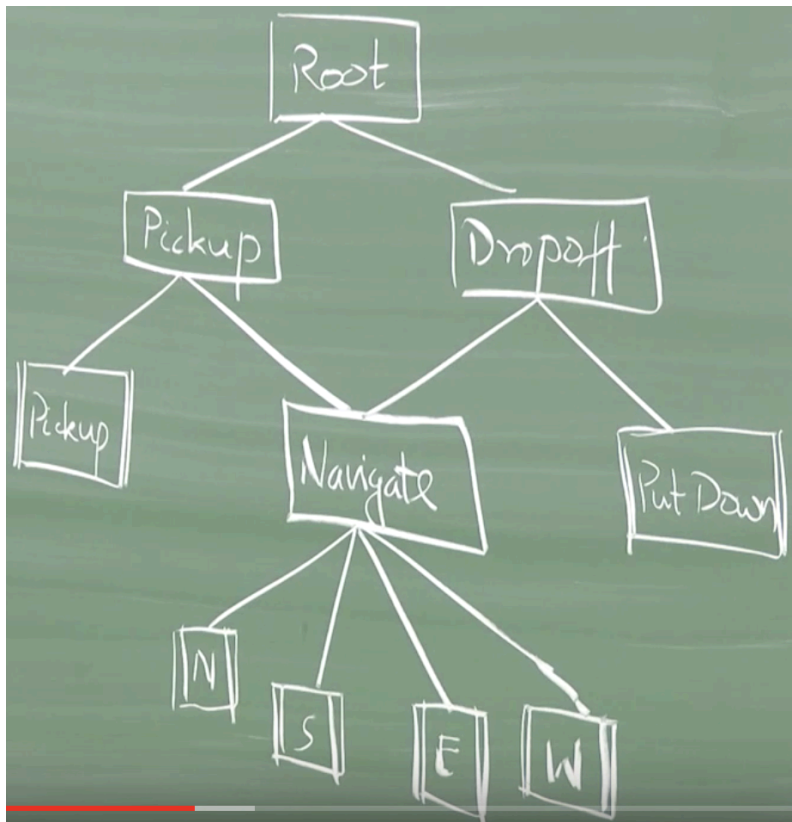
- Go to place where passenger is - sub task - pick up passenger
- Drop him where he wants to get down - sub task - drop of passenger



Double line = primitive actions are -> || Pickup || and || Put Down ||
I need to navigate and then pickup
Go to the destination and then drop off



Expand Navigate:

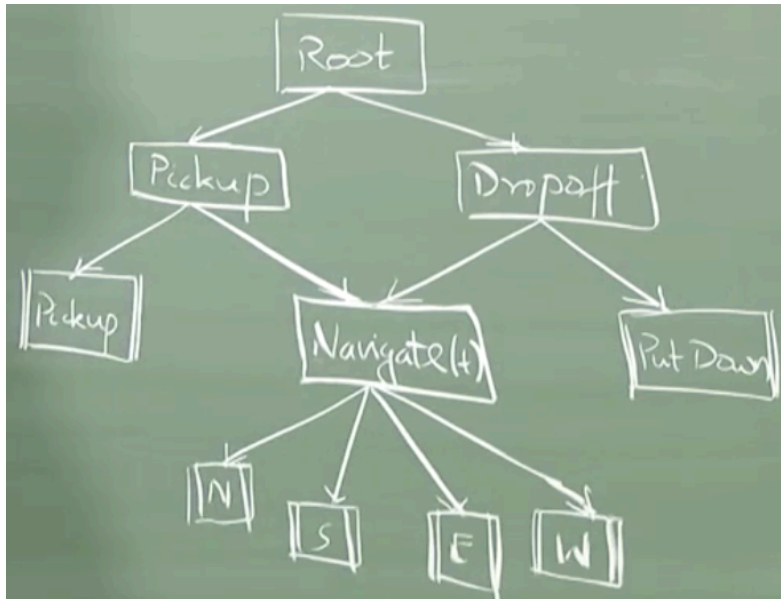


This is like a call graph
Basically - gives clean structure
Little easier than HAM - MAXQ is more intuitive

Navigate needs an argument - $\text{Navigate}(t) \rightarrow t$ is r, g, b, y

- Four different sub tasks

This has to be DAG (Directed Acyclic Graph):



When do i go back to parent $\rightarrow \text{Navigate}(b)$ after that it has to go back to pickup/ dropoff

It doesn't imply any order $\rightarrow$ It is not left to right ordering

- Order is not indication in which the actions will be called

It says - whatever solution i can come up with for Pick up

- I need pickup and navigate

It says - whatever solution i can come up with for Drop off

- I need pickup and navigate

Break in k sub-tasks

For each of the sub-tasks define some quantity

1. Si is sub-task - state variable for taxi

states where pick up tasks can be executed

- Taxi should not have a passenger
- Location of passenger can take = 4, r, g, b, y or in taxi
- State /location of taxi - all locations (25 - 8 obstacles)
- Destination of passenger = r, g, b, y

Successfully

- I can pick up only if location of taxi = location of passenger
- Put down = only if location of taxi = location of passenger

2. Ai will be union of

- all M0, M1, Mk
- six primitive actions
- Can't call back -> how to enforce DAG structure
 - Any sub-task can't call root action M0
 - Ai can't consider any Mi below calling Mi

Ri-bar: pseudo reward:

- Reward associated with sub-task
- If i want to navigate to g, will get some reward

Terminal states:

- Which are end states
- Any thing outside Si
- Navigate(g) is only if i am not at g
- Which are desirable specified by pseudo reward
 - Pick up passenger from pickup location

Navigate:

- Is active as long as i am not in r, g, b, y
- Navigate has to terminate when i reach r, g, b, y
 - It is determined by (navigate(para))

MAQX - has lot of complexity in learning

Options and HAM - You are eventually learning on single MDP	In MAXQ - M0, M1 .. Mk each is SMDP - You are learning them in one shot - Pseudo reward M1, M2 etc for each Mi - Complete task will have M0 reward
---	--

w11L61 MAXQ Value Function Decomposition (42min)

$$T_i U S_i = S$$

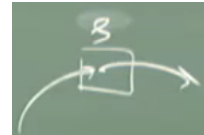
Var is T_i

If you are in a terminal state - you call, it will return

In options: same state can be in i and terminal state

- If i start - will atleast take one action

- While i come back or executing the action, i may terminate



Look at Value Functions:

- Collection of SMDP
- Take one of those M_i - we have S_i, A_i, R_i^*
 - Know A_i and A_i
 - R_i and R together give reward for SMDP

Graph is drawn from A_i

What about T - little tricky - need to define joint distribution

$$\text{Collection of SMDPs.} \\ M_i \\ \langle S_i, A_i, \bar{R}_i + R \rangle$$

NOT correct

$$M_i \\ \langle S_i, A_i, \rangle$$

Correct:

erased reward - it was not correct

- If A_i is primitive action - it is fine core mdp reward
- If i happen to pick non-primitive action
 - What will be my transition
 - Define probability over N (τ) step to complete - no. of transition

$$P_i(s', \tau | s, a)$$

How many times subtask will take to complete

$$p_i(s', \tau | s, a) \sim M_a$$

taks-a

$$R_i(s, a, s', \tau)$$

Ri: you know all about Ri

Summations of rewards you will get
Call a, execute policy - will accumulate reward

Tricky - what should Ri be? From MDP definition

- Reward Expectation
- What i described was sample

Write down:

$$R_i(s, a, s', \tau) = E \left\{ r_t + \gamma r_{t+1} + \dots \mid s_t = s, a_t = a, \pi \right\}$$

Conditioning on S' and tau are tricky, write without conditioning on s' and tau

Defined as rewards for Core MDP:

$$R_i(s, a) = E \left\{ r_t + \gamma r_{t+1} + \dots \mid s_t = s, a_t = a, \pi \right\}$$

Expected Reward $R_i(s, a)$ - already has pi

- Need to know policy below M_i to compute this reward
- a (root) may call a' (pickup), a' may call a'' (navigate), navigate may call south
- And it will return

$\bar{R}_i$ will be defined based on M_i MDP

These transitions will be below me - corresponding to different set of states

- In SMDP below me
- They may correspond to different set of states

Add a separate term for pseudo reward (2nd line below)

$$R_i(s, a) = E \left\{ r_t + \gamma V_{t+1} + \dots \mid x_t = s, a_t = a, \pi \right\}$$

Right now will remove - as it adds lot of complication

Read paper - you have to maintain second value function

- One for only core MDP reward (1st line - coming from sub tasks below me)
- One internally to subtask reward

To successfully completely pickup

- Don't use Navigate

Pseudo reward for root - NO pseudo reward of pickup or dropoff

Pseudo reward is only limited to that subtask

Write value function:

$$V_i^\pi(z, s)$$

value function of state s in task i

How to do Bellman style

$$V_i^\pi(z, s) = R_i^{\pi_i(s)}$$

Cost corresponding to to first action

$$V_i^\pi(z, s) = R_i^{\pi_i(s)} + \sum_{s', z} P_i^\pi(s', z \mid s, \pi_i(s)) \cdot \gamma^z V_i^\pi(z, s')$$

What is missing - summation over pi - why?

Because I am assuming pi is deterministic - thats why wrote pi-s

Say - summation over a is missing - because it is deterministic policy

Have value function corresponding to actual policy π_i

- Looks only till the completion of task - i finishing

$$\begin{array}{c} V^{\pi_i}(s) \\ \Downarrow \\ \text{Projected Value fn.} \\ V^{\pi_i}(i, s) \end{array}$$

- operates with projected function
- Can't learn hierarchical optimal policy
- Others came up for hierarchical optimal or recursive policy

$$V^{\pi_i}(i, s) = R^{\pi_i}(s) + \dots$$

What is this guy in circle:

π_i -subtask

Entire reward's action should go here

$$V^{\pi_i}(i, s) = V^{\pi_i}(\pi_i(s), s) + \sum_{s', z} P_i^{\pi_i}(s', z | s, \pi_i(s)) \cdot \gamma^z \cdot V^{\pi_i}(i, s')$$

Projected value function in π_i -subtask

- Projected value function - reward till end of the task
- Reward accumulated till end of π_i

$$q^{\pi_i}(i, s, a) = V^{\pi_i}(a, s)$$

Similarly can write - in subtask i what is the value function s, a

On left it is s, a on right it is

- a, s (value of s in subtask a)

And will have same thing

$$q^{\pi}(i, s, a) = v^{\pi}(a, s) + \sum_{s', z} p_i^{\pi}(s', z | s, a) \cdot \gamma^z \cdot v^{\pi}(i, s')$$

Write last term in terms of q

$$q^{\pi}(i, s, a) = v^{\pi}(a, s) + \sum_{s', z} p_i^{\pi}(s', z | s, a) \cdot \gamma^z \cdot q^{\pi}(i, s', \pi(s'))$$

Now have Bellman eq for v-pi and q-pi

Define right term - Completion function C -> C-pi (i, s, a) Cost, reward payoff you accumulate

- in subtask i after you complete
- Action a starting from state s

First term is completed action a, i will be in s'

- Will get reward i will get in s' till completion of subtask i

So q function is

$$q^{\pi}(i, s, a) = v^{\pi}(a, s) + \sum_{s', z} p_i^{\pi}(s', z | s, a) \cdot \gamma^z \cdot q^{\pi}(i, s', \pi(s'))$$

$$q^{\pi}(i, s, a) = v^{\pi}(a, s) + \bar{C}^{\pi}(i, s, a)$$

Now in terms of completion function - two terms

$$Q^{\pi}(a, s) = \begin{cases} V^{\pi}(a, s, \pi_a(s)) & \text{if } a \text{ is a "composite"} \\ \sum_{s'} P(s'|s, a) \cdot E\{r|s, a, s'\} & \text{if } a \text{ is primitive} \end{cases}$$

v-pi: Composite action - one of the subtask

v-pi: What is the value of state s under primitive action a - Expected value - marginalised over s'

For composite action to compute, i need C

To compute v - go down and q of the lower action

Some kind of hierarchical thing

$$\pi = \{\pi_0, \pi_1, \pi_2, \dots, \pi_k\}$$

pi- is composed of

Pi-0 to pi-k follow in M0, M1 to Mk sub tasks

Value of state s at zero subtask:

Pi-0 will call some action below - let us say a1

$$Q^{\pi}(a, s) = \begin{cases} V^{\pi}(a, s, \pi_a(s)) \end{cases}$$

pi-0(s) = a1 so write a-pi(a, s, a1)

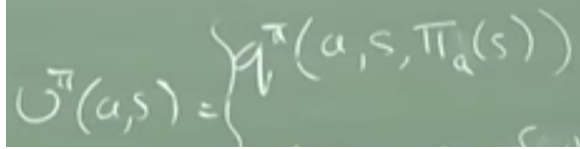
Written as second term below in screen shot

Then add that will be v-pi(a1, s) + C-pi(0, s, a1) - written as first term below

$$Q^{\pi}(0, s) = C^{\pi}(0, s, a_1) + Q^{\pi}(a_1, s)$$

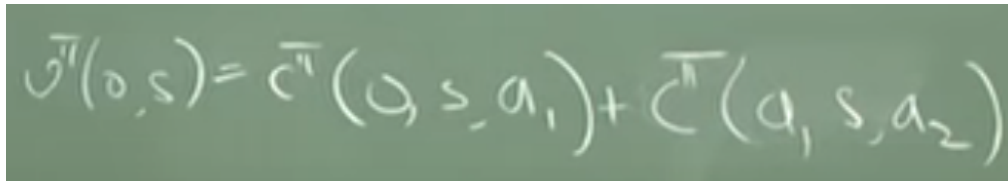
Second action:

Now we go into $a_1 \rightarrow$ 1st action I take is a_2 (composite action)


$$V^{\pi}(a, s) = q^{\pi}(a, s, \pi_a(s))$$

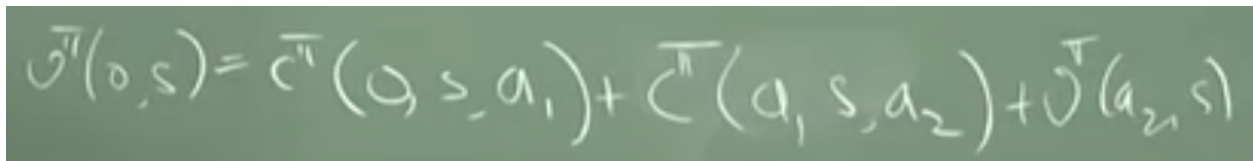
$v\text{-}\pi(a_1, s)$ will be $q\text{-}\pi(a_1, s, a_2)$

Go back to $q\text{-}\pi(a_1, s, a_2) = v\text{-}\pi(a_2, s) + C\text{-}\pi(a_1, s, a_2)$


$$V^{\pi}(0, s) = C^{\pi}(0, s, a_1) + C^{\pi}(a_1, s, a_2)$$

+

Conditioned on whole policy - π
 $\pi\text{-}o(s)$ is a_1


$$V^{\pi}(0, s) = C^{\pi}(0, s, a_1) + C^{\pi}(a_1, s, a_2) + V^{\pi}(a_2, s)$$

Will keep going until I hit a primitive action

For $v\text{-}\pi(0, s) = \text{completion action} + \text{choice made in sub task} + \text{keep going}$

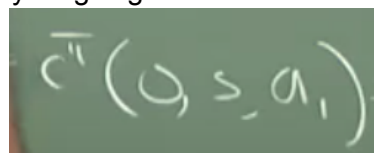
This is value function decomposition

Suppose I look at $v\text{-}\pi(0, s') \rightarrow$

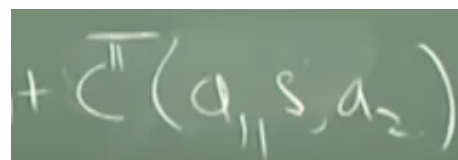
- Further down the hierarchy can reuse the completion function
- And can reuse q function
- Break down the value function into chunks of rewards, which I am likely to see again and again
- Each chunk corresponds to a sub task

This is complex - w/o pseudo rewards

If you look at root task - everything is good


$$C^{\pi}(0, s, a_1)$$

- Completion function for root task


$$+ C^{\pi}(a_1, s, a_2)$$

- Go to sublevel - only for completing a_1
- Completion fn - don't care what happens after a_1

- I need to know what happens after completion of a_1 (get out of the door)

What was proposed was - second completion function

- Complex
- How do you do efficient book keeping

Now you can define SMDP Q-learning

- Update q value of subtask until i reach primitive action

Primitive ask i don't know -

$$\sum_{s'} P(s'|s, a) \cdot E\{r|s, a, s'\} \text{ if } a \text{ is primitive}$$

Maintain some kind of running average -

- and you return running average - it is q value (taking action a in state s)

$$E\{r|s, a, s'\}$$

- q value will have future expectations

2nd thing to remember: for every action remember completion task

$$Q^{\pi}(0, s) = c^{\pi}(0, s, a_1) + Q^{\pi}(a_1, s, a_2) + Q^{\pi}(a_2, s)$$

Update completion cost every state - sub task

At primitive action - update expected reward

MAXQ(0) learning : pseudo rewards is zero (which i described)

- Uniformly all subtasks in all states

MAXQ(Q) : arbitrary pseudo rewards. Maintain two completion fn

- One for solving the subtasks - picking actions in the subtasks
-
- Other for answering queries from above
 - When pickup asks navigate give me your completion function
 - Navigate will send the function which was learnt with core rewards
 -
- When navigate wants to know the q value to pick an action
 - It will use completion function learnt with pseudo reward

Book keeping becomes painful

w11L62 Option Discovery (16min)

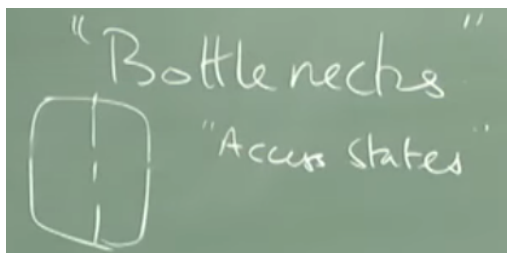
What will be good option to discover?

- Reusability
- Cut down on exploration
 - How will you find out
- Transfer learning

Instead of actual thing, use surrogate measure

1st thing: **Bottlenecks (= access gates)**

- Useful heuristics
- Something that is limiting - it has funnel kind of effect
- Example: Doorways



Exploration - if know effectively can get out

Reusability - use learning of how to behave in two states (1st half of mdp and 2nd half of mdp)

- Even while learning - even if you move goal around

Transfer learning - even if you move goal around - knowing bottleneck is useful

How do you find bottleneck

If I give you fi (transition structure of an mdp) how will you find out bottleneck

Try to segment the mdp - model as graph

- Two states are connected if one action takes from one to another'
- Find graph cuts - find components which are weakly connected
- Those states where weak connection happens will be your bottleneck states

1st approach: Graph Partitioning idea:

- Graph clustering to find different components

What if you don't have graph (mdp):

- Walk around mdp randomly
- Form mdp

2nd approach : Diverse Density:

- Many trajectories generated solving (successful trajectories)
- Many trajectories generated aborted (unsuccessful trajectories)

States appear disproportionately more in successful trajectories

- Keep track of count (few 100s or 1000s trajectories)

Diverse density - they do it through experience replay

- Replay those trajectories and learn q function
- Option policy for going to those states

1st option discovery approach:

- Most implementable approach
- However small data you give, it throws options

3rd approach: Betweenness:

In graph - shortest path goes from fewer states

- High betweenness = bottleneck
- You can define bottleneck in terms of betweenness
- Run betweenness on graph - find states
- I , π and β
 - Bottleneck state gives β

Did a fun project - in social networks course

4. Small world option (random option):

- Convert mdp to small world mdp
- Inserted options at random into the world
- The expected length of transition of the options followed certain probability distribution
 - Need to convert mdp into small world mdp
 - Small world mdp - you reach from one node to another node in small no. of options
 - N is size of graph ($\log N$)-square hop
- There are results:
 - If you manage to insert edges in a graph that has certain length distribution
 - Can convert them into small world option
 - That cuts down exploration
 - Improves learning time effectively
- Any no. of random trajectories you give us
 - Can define useful option

Data equivalence:

- Small world option makes better use of data
- Compared to betweenness and bottleneck
- It should do well in transfer learning
 - Depends on dynamics of problem
 - Not on rewards
- Reusable - high - connects different points in the world